

Pre-PGI: What Your System Requires Us To Build

Findings from running the actual functions in your DS4 and QS4 systems — and the design decision that follows from them.

SYSTEMS	METHOD	DATE	DECISION
DS4 / 200 QS4 / 700	Live execution, not documentation	19 Aug 2026	One API for the pre-PGI span

THE DECISION

One API covering shipment, freight cost and cost release. Not three.

Not a preference — **four of the six steps have no interface at all**, so custom SAP code is required whichever way we go.

Every figure and document number in this paper was read from your system. We ran the functions rather than reading about them.

1 What your system actually does

The portal shows one button. SAP performs **six things, one after another**. Each one saves separately — once a step is saved, it is saved. You do not undo it; you reverse it with another document.

1	Create the shipment	The truck run
2	Create the freight cost document	The freight bill
3	Calculate it	Work out the money
4	Account assign it	Decide which account it hits
5	Settle it	Push the cost into purchasing — this creates a real PO and service entry sheet
6	Goods issue	Stock leaves the plant

What everyone calls "cost release" is step 5. It is not a tickbox. It is SAP pushing the freight cost into Materials Management. That is why a purchase order appears when it works.

A real example from your system

Delivery **9004953084** , 17 August, in your quality system:

Time	What happened
10:26:23	Delivery created
11:11:28	Shipment 1100605471 created
11:11:28	Freight cost document 1100608871 created and calculated — 7,460.00 INR
11:11:30	Settled → purchase order 6000105831 , service entry 1003382363
11:11:31	Goods issue
11:11:32	Invoice

Shipment to settled took **two seconds**. This is a fast chain, not a long-running process.

2 Why custom code is unavoidable

To call something in SAP from outside, SAP has to provide a door — a published interface. We checked every step.

Step	Can we call it from outside?
1 · Create the shipment	YES
2 · Create the freight cost document	NO
3 · Calculate	NO
4 · Account assign	NO
5 · Settle	NO
6 · Goods issue	YES

The conclusion

Four of the six steps have no interface.

Custom SAP development is therefore **required**, regardless of how many endpoints anyone prefers. This is a property of your system, not a design opinion. **"Use standard APIs instead" is not available for this stretch.**

How we know: we searched SAP's function registry by name pattern, and asked SAP directly which programs call which functions — each search with a control test to prove the search itself was working. We state this as a finding from those searches, not as a claim about the whole universe of SAP.

3 What we proved by running it

Three things were open questions. We settled all three by executing against your DS4 system.

3.1 The internal functions *can* be driven from custom code

This was the real risk. If those four internal steps only worked inside their own screen, nothing would be buildable.

We created a real shipment in DS4 and then called the internal freight-cost function directly against it, outside its normal transaction. **It ran clean in three seconds, accepted its input and rejected nothing.**

Risk closed

The wrapper approach works. This was the single assumption the whole design rested on.

3.2 Your own team already built this pattern

When their developers needed to automate freight costing, they hit exactly the same wall and solved it the same way. There is a function in your system today that takes a shipment number, a cost type and a save flag, and calls the same internal SAP function we intend to call.

We are not inventing an approach. We are building the CNF version of something already running in your landscape.

The one difference

Theirs returns **nothing** — no document number, no error message. It cannot tell its caller what it created or why it failed. That is fine behind a screen where a person is watching. It is not fine for an interface. **Ours must report back.**

3.3 The shipment interface does not save by itself

We ran the standard shipment-create function twice with identical input.

What we did	What SAP returned	What actually existed
Created the shipment, nothing else	Shipment 1000, "Save shipment", no errors	Nothing. The table was empty.
Created it, then issued the save step	Shipment 1001	Shipment 1001 exists.

Three consequences

1. Whoever calls it must issue the save. If they don't, everything disappears.
2. SAP gives you the number *before* it saves. So you can hand the portal "shipment 1000" for a shipment that never existed — and number 1000 is now permanently used up.

3. Every message came back green. Success does not mean saved.

This is exactly how orphaned records appear and go unnoticed. Our service handles the save so no caller has to remember.

4 Four things worth correcting

Found while we were in the system. Each one would cause a real problem if built on.

1 · The field everyone reads as "cost release" is not cost release

It is the **account assignment** status. And blank on it means "not applicable" — not "not done". So it never indicated a missed step. There is no release field on the freight document at all; release is the settlement step.

2 · The freight document number is not the shipment number

They look alike and sit in similar ranges, but they are different documents. Anything built by matching them will quietly give wrong answers.

3 · Cost type is required, and it is missing from the specification

SAP will not create a freight document without one. We found four different cost types in your system behaving differently — one completes the full cycle, another calculates and then stops. **Somebody needs to decide whether the portal sends it or we derive it.**

4 · Billing cannot be created through the billing API

That service is read-only for creation. If any part of the design assumes otherwise, it needs changing now rather than in testing.

5 What we will build

One function, called once by CNF. It runs SAP's own steps in order, saves them, and reports exactly what was created.

What goes in

What the C&F agent's screen already collects — delivery, transporter, vehicle number, driver name and mobile, LR number and date, the three weights, route, shipment type and planning point — plus the cost type discussed above.

The vehicle, driver, LR and weight fields already have a home: they are custom fields your team added to SAP's standard shipment structure. We simply pass them through.

What it does inside

#	Step	Whose code
1	Check what already exists	Ours — a few reads
2	Create the shipment	SAP's own
3	Create and calculate the freight cost	SAP's own
4	Settle it	SAP's own
5	Save	SAP's own
6	Read back and report what exists	Ours — a few reads

We add no freight logic. No rates, no routes, no accounting. All of that stays SAP's, using your existing configuration. We add a door, the save, and a truthful answer.

What comes back

Shipment number, freight document number, cost type, **which step it reached**, the purchase order and service entry if settlement worked, and SAP's own error messages if it did not.

The important part is **which step it reached** — not a yes/no. If something fails at settlement, CNF is told: shipment exists, freight document exists, settlement failed, here is SAP's message. Nobody guesses. Nobody goes digging.

Why it is safe to call twice

If a call times out and CNF does not know what happened, it simply sends the same request again. The service looks first: shipment already there? freight document already there? settled? It skips what is done and finishes the rest.

No duplicate shipment. No duplicate freight cost. This only works because the truth lives in SAP's tables — which is also the main reason for one call rather than three.

6 Why one call, not three

Each step needs the number the previous step just created

SAP invents the shipment number. Step 3 cannot start without it. SAP then invents the freight document number, and step 5 cannot start without *that*.

Split into three calls and your portal has to catch SAP's shipment number, hold it, hand it back for the next call, catch the next number, hand that back again. **With one call those numbers never leave SAP.**

Only SAP knows how far a run got

The progress lives in SAP's tables. Three separate calls mean something outside SAP tracking state it cannot see. One call reads those tables directly.

SAP itself refuses to split two of the steps

Creating and saving the freight document is a single operation inside SAP — one calls the other. Exposing them separately would invent a boundary SAP does not have.

And the number worth knowing

10,496 freight cost documents in your quality system are stuck — calculated, meant to be settled, never were. No purchase order, no service entry. The deliveries still shipped and still got invoiced. Only the freight cost never reached the books.

That is **0.18%** of 5.7 million documents. Small — and we want to be straight about that. But:

Why it still matters

That 0.18% is the **best case**. It is SAP code, running inside SAP, with no network between the steps, tuned over years. It still leaves 10,496 behind. Splitting these steps across a network, with something outside SAP tracking them, will not do better.

At around 4,000 logistics calls a day, that is roughly **7 a day, 2,600 a year**.

7 What we are asking for

Say this plainly now, not later

Not all-or-nothing
Once the shipment is saved, it is saved. If settlement fails you do not rewind — you reverse. This is equally true of one API or three, and it cannot be designed away. **The word "rollback" used in the last session should be withdrawn.**

Two things we need from you

What	Why
A test setup in DEV — one transporter, one material with stock, one delivery	DS4 currently has no transporters at all , which blocks everything from the shipment onwards. We cannot run the full chain end to end until this exists.
One process answer — which transaction or job your team actually runs to create and settle freight costs	No amount of technical inspection produces this. Only your functional people know it.

Both are build prerequisites, not gaps in the analysis.

One more decision for you

Your CNF-specific document types are not present in the development system. Two options:

	Prove the mechanics now	Prove your exact setup
Uses	Standard SAP document types	Your own types brought into DEV
Proves	The build works — payloads, saving, retry	Actual CNF behaviour
Speed	Fast, once test data exists	Needs a configuration transport

We recommend the first now, the second before sign-off. We will never present results from standard types as proof of your configuration.

In one line
Your system forces custom code here — four of six steps have no interface.
Your own team already built this exact pattern once. We are asking to build it properly for CNF: **one call, it tells you what it created, and it is safe to call twice.**

Prepared from live execution in DS4/200 and read-only analysis of QS4/700. Document traces, captured interfaces and full counts available on request.